SIGNITC: Supersingular Isogeny Graph Non-Interactive Timed Commitments

Knud Ahrens
University of Passau, Germany
knud.ahrens@uni-passau.de

Abstract

Non-Interactive Timed Commitment schemes (NITC) allow to open any commitment after a specified delay t_{fd}. This is useful for sealed bid auctions and as primitive for more complex protocols. We present the first NITC without repeated squaring or black box algorithms like generic NIZK proofs or generic one-way functions. It has fast verification, almost arbitrary delay and satisfies IND-CCA hiding and perfect binding. Our protocol is based on isogenies between supersingular elliptic curves making it presumably quantum secure. It needs no trusted setup and can use a wide variety of primes. We also have a SageMath implementation.

Keywords: Non-interactive timed commitments, post-quantum, isogeny walks, Deuring correspondence.

1 Introduction

The concept of time-lock puzzles (TLP) [32] has been around for more than twenty years and timed commitments [7] developed shortly after. While a TLP only provides an encryption such that decryption takes at least a specific amount of time, timed commitments also specify verification algorithms. We will use the rather new definition of Non-Interactive Timed Commitment schemes (NITC) by Katz, Loss, and Xu [27] from the year 2020. These protocols satisfy binding or non-malleability properties and efficient verification just like usual commitment schemes, but a commitment can be opened by anyone after some delay t_{fd}. So hiding only lasts for this time t_{fd} and there are additional algorithms: one to verify that a commitment can be opened by others and another one to open the commitment forcefully in time at least t_{fd}. A possible application is a sealed bid auction, where all bids can be revealed after time t_{fd} even if some of the bidders refuse to open their commitment. Other applications include e-voting, fair coin tossing or contract signing [7]. Since we can build a TLP from a NITC by using the commitment as encryption and the forced decommitment as decryption, we can use (non-interactive) timed commitments for all applications of the broader time-lock puzzles.

Our contribution is (to our knowledge) the first quantum secure NITC scheme with explicit algorithms. We use random walks in the isogeny graph of supersingular elliptic curves to construct a NITC, hence the name Supersingular Isogeny Graph Non-Interactive Timed Commitments or SIGNITC[1] for

[1] pronounced like "signets"

short. The main idea is that computing isogenies of large or non-smooth degree is slow, but if we know the endomorphism ring of the starting curve, we can find a smooth shortcut. So we use a secret isogeny to a curve with known endomorphism ring for fast commitment and verification, but the forced decommitment has to compute the delay isogeny and thus it needs time at least t_{fd}.

The advantage of isogeny-based cryptography is that it is presumably quantum secure and relatively slow compared to other fields of post-quantum cryptography. This is good for the delay, but can be a challenge for the verification. The field has undergone thorough scrutiny due to the candidates SIKE [26] and SQISign [19] in NIST competitions for post-quantum protocols and is still very active. The protocol only uses (known) isogeny-based cryptography, so we do not need to know several fields and this facilitates correct and secure implementations. This also means that we have no generic zero-knowledge proofs, succinct non-interactive arguments (SNARGs) of knowledge or one-way functions that are treated as black box algorithms. An implementation in SageMath is available on GitHub[2]. The only drawbacks are that some algorithms are still quite involved and that we need to slightly modify the security game for hiding to adapt them to our setting.

Related Work Thyagarajan et al. [35] present an approach based on class groups using non-interactive zero-knowledge (NIZK) proofs. Katz et al. [27] and Chvojka and Jager [14] use protocols based on repeated squaring in a group of unknown order and NIZK proofs. Finally, Ambrona et al. [2] avoid NIZK proofs but still use repeated squaring. None of these is quantum secure.

NITC schemes are related to verifiable delay functions (VDF) [8] in the sense that both have fast verification and a function that needs a long time to evaluate. The main difference is the handling of secrets. For VDFs finding the correct response for a given challenge has to be slow for everyone. For NITC schemes however someone has to compute the commitment and therefore already knows the output of the slow task, namely finding the message to a given commitment. So we can construct NITC schemes from VDFs, but the contrary is difficult or impossible, depending on the protocol.

VDFs have direct applications to blockchains and there are already several approaches. Many are based on repeated squaring in groups of unknown order for the delay. A new publication [6] suggests that repeated squaring modulo a known prime might not be sequential. So contrary to current belief, repeated squaring could be parallelizable in some settings, disqualifying it as a delay function. Additionally, this is not quantum secure. There are even some isogeny-based candidates for VDFs, but they all still have some flaws. The pairing-based approach [18] is not quantum secure. Chavez-Saab et al. [12] use SNARGs and their verification time increases for larger delays. Finally, there is one base on Kani's criterion for abelian surfaces [21], but the authors state that it is not clear how to implement it. A different approach based on endomorphism rings [1] has the problem that the generation of a challenge also gives (a significant advantage in finding) the response. So it is closer to a NITC scheme and gave the initial idea for this article.

Burdges and De Feo [9] introduced isogeny-based delay encryption, but they use the same delay as the pairing-based VDF. Sterner [34] presented two com-

[2] https://github.com/kahrens-code/signitc

mitment schemes using isogenies, but they both require a trusted setup and have no algorithm for opening them after some time.

Structure of this Article The remainder of this paper is structured as follows. First we give a definition of NITC schemes and discuss their properties. Next we recall the necessary definitions and fix the notations of isogeny-based cryptography. Readers familiar with one of these topics can briefly skim through the respective sections as we aimed to use standard notations. The sole difference is a slight variation in Definition 5.13 of IND-CCA security games. In Section 4 we present our protocol in full detail. Its security and its properties are discussed in Section 5. Finally, we present some benchmarks in Section 6 and give a short conclusion.

2 Non-Interactive Timed Commitments

In this section we recall NITC schemes and their properties. In their paper Katz et al. [27] gave the first formal definition of this concept.

Definition 2.1 (NITC [27])**.** *A* $(t_{\mathrm{com}}, t_{\mathrm{cv}}, t_{\mathrm{dv}}, t_{\mathrm{fd}})$ non-interactive timed commitment scheme (NITC) *is a tuple* $\mathtt{TC} = (\mathtt{PGen}, \mathtt{Com}, \mathtt{ComVrfy}, \mathtt{DecVrfy}, \mathtt{FDecom})$ *of five algorithms with the following behavior:*

- *The randomized parameter generation algorithm* `PGen` *takes as input the security parameter* 1^κ *and outputs a common reference string* **crs**.
- *The randomized commit algorithm* `Com` *takes as input a string* **crs** *and a message* m*. It outputs a commitment* $\mathbf{C}$ *and proofs* π_{com}*,* π_{dec} *in time at most* t_{com}.
- *The deterministic commitment verification algorithm* `ComVrfy` *takes as input a string* **crs***, a commitment* $\mathbf{C}$ *and a proof* π_{com}*. It outputs* **accept** *(if* $\mathbf{C}$ *could be forcefully decommitted) or* **reject** *in time at most* t_{cv}.
- *The deterministic decommitment verification algorithm* `DecVrfy` *takes as input a string* **crs***, a commitment* $\mathbf{C}$*, a message* m *and a proof* π_{dec}*. It outputs* **accept** *or* **reject** *in time at most* t_{dv}.
- *The deterministic forced decommitment algorithm* `FDecom` *takes as input a string* **crs** *and a commitment* $\mathbf{C}$*. It outputs a message* m *or* **invalid** *in time* at least t_{fd}.

We require that for all κ*, all* **crs** *output by* $\mathtt{PGen}(1^\kappa)$*, all* m *and all* $\mathbf{C}, \pi_{\mathrm{com}}, \pi_{\mathrm{dec}}$ *output by* $\mathtt{Com}(\mathbf{crs}, m)$*, it holds that*

$$\mathtt{ComVrfy}(\mathbf{crs}, \mathbf{C}, \pi_{\mathrm{com}}) = \mathbf{accept} = \mathtt{DecVrfy}(\mathbf{crs}, \mathbf{C}, m, \pi_{\mathrm{dec}})$$

and $\mathtt{FDecom}(\mathbf{crs}, \mathbf{C}) = m$.

To be relevant for applications a NITC also needs to satisfy three further properties. First we give a definition of *practicality* and then recall definitions for *hiding* and *binding* in our notation. There is no benefit in verification algorithms if we can verify faster by using `FDecom`. There are applications for protocols

where Com is as slow as FDecom, but efficiently creating commitments even for long delays is desirable. Definition 2.2 tries to represent this. A more precise definition is difficult as one may wish for an absolute gap for small delays and relative gap for large delays.

Definition 2.2 (Practicality)**.** *A NITC scheme is* practical*, if verification is much faster than forcefully opening the commitment, so* $t_{\mathrm{cv}}, t_{\mathrm{dv}} \ll t_{\mathrm{fd}}$*. If in addition the commitment is also much faster than forced decommitment, i.e.* $t_{\mathrm{com}} \ll t_{\mathrm{fd}}$*, we call it* perfectly practical*.*

We present two IND-CCA security games and define hiding in terms of the probability that an adversary $\mathcal{A}$ wins the games. In both cases the adversary has access to an oracle for FDecom and a query is considered to have only a small computational cost. The first game is the one used by Katz et al. [27].

Definition 2.3 (IND-CCA original [27])**.** *For a NITC scheme* TC *and an algorithm* $\mathcal{A}$*, define the game* $\texttt{IND-CCA}^{\mathcal{A}}_{\texttt{TC}}$ *as follows:*

1. *Compute* $\mathbf{crs} \leftarrow \texttt{PGen}(1^{\kappa})$.
2. *Run* $\mathcal{A}(\mathbf{crs})$ *in a preprocessing phase with access to* $\texttt{FDecom}(\mathbf{crs}, \cdot)$.
3. *When* $\mathcal{A}$ *outputs* (m_0, m_1)*, choose a uniform bit* $b \leftarrow \{0,1\}$ *and then compute* $(\mathbf{C}_b, \pi_{\mathrm{com}}, \pi_{\mathrm{dec}}) \leftarrow \texttt{Com}(\mathbf{crs}, m_b)$*. Give* $(\mathbf{C}_b, \pi_{\mathrm{com}})$ *to* $\mathcal{A}$*, who continues to have access to* $\texttt{FDecom}(\mathbf{crs}, \cdot)$ *except that it may not query the oracle on the given commitment* $\mathbf{C}_b$*.*
4. *When* $\mathcal{A}$ *outputs a bit* b'*, it wins iff* $b' = b$*.*

The commitment $\mathbf{C}$ in our approach is a tuple and not a single value. Because of that we can only satisfy a slightly weaker variation of the IND-CCA security game. The new Definition 5.13 is given and discussed in Section 5.2. Hiding is defined with respect to an IND-CCA game. This allows us to evaluate the security of our NITC in terms of both the original and our adapted definition. Broadly speaking hiding guarantees that it is impossible to infer information about the message from the commitment. In our case hiding should hold at least for the time t_{fd} it takes to open a commitment by force, so for all $t_o < t_{\mathrm{fd}}$ in the following definition.

Definition 2.4 (Hiding [27])**.** *A NITC scheme* TC *is* (t_p, t_o, ε)-CCA-secure *if for all adversaries* $\mathcal{A}$ *running in time at most* t_p *in the preprocessing phase and time at most* t_o *in the subsequent online phase,*

$$\Pr\left[\mathcal{A} \text{ wins } \texttt{IND-CCA}^{\mathcal{A}}_{\texttt{TC}}\right] \leq \frac{1}{2} + \varepsilon.$$

Similar to hiding, binding is defined in terms of the probability that $\mathcal{A}$ wins a BND-CCA security game. This time we do not need to adapt it for our approach.

Definition 2.5 (BND-CCA [27])**.** *For a NITC scheme* TC *and an algorithm* $\mathcal{A}$*, define the game* $\texttt{BND-CCA}^{\mathcal{A}}_{\texttt{TC}}$ *as follows:*

1. *Compute* $\mathbf{crs} \leftarrow \texttt{PGen}(1^{\kappa})$.

2. *Run* $\mathcal{A}(\mathbf{crs})$ *with access to* $\texttt{FDecom}(\mathbf{crs}, \cdot)$.

3. $\mathcal{A}$ *outputs* $(m, \mathbf{C}, \pi_{\text{com}}, \pi_{\text{dec}}, m', \pi'_{\text{dec}})$ *and wins iff* $\texttt{ComVrfy}(\mathbf{crs}, \mathbf{C}, \pi_{\text{com}}) = \mathbf{accept}$ *and either:*

   - $m \neq m'$*, yet* $\texttt{DecVrfy}(\mathbf{crs}, \mathbf{C}, m, \pi_{\text{dec}})$ *and* $\texttt{DecVrfy}(\mathbf{crs}, \mathbf{C}, m', \pi'_{\text{dec}})$ *both output* $\mathbf{accept}$*;*
   - $\texttt{DecVrfy}(\mathbf{crs}, \mathbf{C}, m, \pi_{\text{dec}}) = \mathbf{accept}$ *but* $\texttt{FDecom}(\mathbf{crs}, \mathbf{C}) \neq m$.

Binding makes sure that a commitment can not be opened to two different messages and that FDecom gives the correct messages for valid commitments.

Definition 2.6 (Binding [27])**.** *A NITC scheme* TC *is* (t, ε)-BND-CCA-secure *if for all adversaries* $\mathcal{A}$ *running in time* t*,*

$$\Pr\left[\mathcal{A} \text{ wins } \texttt{BND-CCA}_{\texttt{TC}}^{\mathcal{A}}\right] \leq \varepsilon.$$

The original definition of a NITC [27] does not specify the allowed parallelism. In Section 2 of that paper they allow all algorithms to have arbitrary parallelism, but this probably does not include the section on NITCs. Similar to the argument for VDFs [8], unbounded parallelism would allow an adversary to compute all possible commitments simultaneous and hence break hiding. Therefore, we need to bound the allowed parallelism for an adversary. VDFs can require up to $\text{poly}(\kappa, \log t_{\text{fd}})$ cores to achieve the desired delay t_{fd} and weak VDFs can even require $\text{poly}(\kappa, t_{\text{fd}})$ cores. Both have to satisfy (p_f, σ)-sequentiality[3]. Roughly speaking, no algorithm running in time $\sigma(t_{\text{fd}}) < t$ on $p_f(t_{\text{fd}})$ cores can compute a correct response with non-negligible probability. Since NITCs only require FDecom to run in time at least t_{fd} we do not need a minimum of parallelism to be fast enough, but we still have to bound the parallelism of an adversary. This choice then influences all mentioned times in the above definitions.

3 Isogeny-based Cryptography

In this section we provide the necessary basics for isogeny-based cryptography, quaternion algebras and the Deuring correspondence. We also discuss some computational problems in this area.

3.1 Elliptic Curves and the Quaternion Algebra

Elliptic curves have ties to different fields resulting in several equivalent definitions. We will mostly follow the notation of Silverman [33], but restrict ourselves to aspects relevant for this paper.

Definition 3.1 (Elliptic Curve)**.** *An* elliptic curve *is a pair* (E, ∞)*, where* E *is a curve of genus one and* $\infty \in E$*. It is* defined over a field K*, if it is defined over* K *as a curve and* $\infty \in E(K)$*.*

We can define an addition of points on the curve making $(E, +)$ an additive group where ∞ is the neutral element. This permits scalar multiplication written as $[m]\colon E \to E$ and torsion subgroups $E[m] := \{P \in E \mid [m]P = \infty\}$.

[3]The $\sigma(t_{\text{fd}})$ for VDFs corresponds to t_o in Definition 2.4 for NITCs.

Definition 3.2 (Isogeny)**.** *Let E and E' be elliptic curves. Then a morphism $\varphi\colon E \to E'$ such that $\varphi(\infty) = \infty$ is called an* isogeny*. If a non-zero isogeny $\varphi\colon E \to E'$ exists, then E and E' are called* isogenous*.*

In fact, every isogeny between two curves is also a group homomorphism. The isogenies from a curve E into itself form the endomorphism ring $\operatorname{End} E$. Isogenies can be written as rational maps and their degree is defined by this map. Thus, the degree $\deg(\varphi \circ \varphi') = \deg\varphi \deg\varphi'$ is multiplicative. In addition, each non-zero isogeny $\varphi\colon E \to E'$ has a unique dual isogeny $\hat{\varphi}\colon E' \to E$ such that the composition $\hat{\varphi} \circ \varphi = [\deg\varphi]$ is the multiplication by the degree. The isogenies of degree 1 are the isomorphisms, and each isomorphism class can be labelled by the so-called j-invariant. This allows to construct the ℓ-isogeny graph that has those j-invariants as vertices and isogenies of degree ℓ as edges.

Definition 3.3 (Supersingularity)**.** *Let K be a field of characteristic $p > 0$ and E an elliptic curve defined over K. The curve E is* supersingular *if the torsion group $E[p]$ is trivial. Equivalently, this means that the endomorphism ring $\operatorname{End} E$ is an order in a quaternion algebra.*

For the rest of this paper $p > 3$ will be a large prime. This allows us to write every elliptic curve in short Weierstraß form as $E\colon y^2 = x^3 + Ax + B$ with $j(E) = 108(4A)^3/(4A^3 + 27B^2)$. For supersingular curves there is always a representation with A, B, j in $\mathbb{F}_{p^2}$. The Galois conjugate curve $E^p\colon y^2 = x^3 + A^p x + B^p$ has $j(E^p) = j^p$. There are only $\lfloor p/12 \rfloor + \varepsilon$ supersingular elliptic curves for fields with characteristic p where $\varepsilon \in \{0, 1, 2\}$. Hence, the set of supersingular j-invariants has cardinality at least $\lfloor p/12 \rfloor$.

We have already seen in Definition 3.3 that supersingular curves are related to quaternion algebras. We are interested in the quaternion algebra $\mathcal{B}_{p,\infty}$ ramified at p and infinity with $\mathbb{Q}$-basis $\{1, \mathrm{i}, \mathrm{j}, \mathrm{k}\}$ such that

$$\mathrm{i}^2 = -1, \quad \mathrm{j}^2 = -p, \quad \mathrm{k} = \mathrm{ij} = -\mathrm{ji}.$$

The (reduced) norm of an element $\alpha = a_0 + a_1\mathrm{i} + a_2\mathrm{j} + a_3\mathrm{k} \in \mathcal{B}_{p,\infty}$ is given by $\operatorname{nrd}(\alpha) = \alpha\overline{\alpha}$ for $\overline{\alpha} = a_0 - a_1\mathrm{i} - a_2\mathrm{j} - a_3\mathrm{k}$. The bilinear form $f(\alpha, \beta) = (\alpha\overline{\beta} + \beta\overline{\alpha})/2$ satisfies $f(\alpha, \alpha) = \alpha\overline{\alpha} = \operatorname{nrd}(\alpha)$, and two elements $\alpha, \beta \in \mathcal{B}_{p,\infty}$ are called orthogonal if $f(\alpha, \beta) = 0$. An order in $\mathcal{B}_{p,\infty}$ is a lattice that is also a subring, and it is maximal if its discriminant equals p. Now an elliptic curve E is supersingular if and only if $\operatorname{End} E$ is isomorphic to a maximal order $\mathcal{O}$ in $\mathcal{B}_{p,\infty}$.

Theorem 3.4 (Deuring Correspondence [22])**.** *The isomorphism classes of supersingular elliptic curves correspond to the isomorphism classes of invertible left $\mathcal{O}$-ideals in the quaternion algebra, for a fixed maximal order $\mathcal{O}$.*

This so-called Deuring correspondence also gives us that an ℓ-isogeny φ starting at E corresponds to a left ideal I_φ of norm ℓ in $\mathcal{O} \cong \operatorname{End} E$, and the image curve has an endomorphism ring isomorphic to the right order $\mathcal{O}_R(I_\varphi) = \{\alpha \in \mathcal{B}_{p,\infty} \mid I_\varphi\alpha \subseteq I_\varphi\}$ of I_φ, see [36, Ch. 42] for more details.

3.2 Application to Cryptography

Many isogeny-based protocols rely on secret walks in isogeny graphs of supersingular elliptic curves. The fact that the endomorphism ring is non-commutative

gives rise to presumably quantum secure protocols. Moreover, the graphs have fast mixing properties, meaning that we reach an almost uniform distribution on the graph after a short random walk [17].

Taking n steps in the ℓ-isogeny graph corresponds (up to isomorphism) to an isogeny $\varphi\colon E \to E'$ of degree $d = \ell^n$. For our purposes the degree of such isogenies will always be coprime to the characteristic p of the field and the isogeny φ is determined by a point K of order d on the starting curve E. This point generates the kernel of φ and we write $E' \cong E/\langle K\rangle$. In this case the d-torsion group $E[d]$ has d^2 elements and can be generated by two points P, Q of order d on E. This allows us to efficiently choose and describe a random walk by two integers a, b such that $K = [a]P + [b]Q$. We can even use this to define a random walk starting on a different curve. For an isogeny $\psi\colon E \to E''$ with degree coprime to the degree of φ the pushforward $[\psi]_\star\varphi$ is determined by the kernel $\langle\psi(K) = [a]\psi(P) + [b]\psi(Q)\rangle$ and starts at the codomain E'' of ψ. Note that although every supersingular elliptic curve has a representation in $\mathbb{F}_{p^2}$, the kernel of an isogeny and hence its generators might be elements of extensions $\mathbb{F}_{p^{2e}}$.

Now we list some computational tasks that are relevant for isogeny-based cryptosystems. First we present tasks that can be solved efficiently and have a polynomial or even constant complexity.

Task 1: Compute isogenies given their kernels.

Task 2: Given two elliptic curves E, E', an isogeny $\varphi\colon E \to E'$ as well as the corresponding order $\mathcal{O} \cong \operatorname{End} E$ and ideal I_φ, compute $\mathcal{O}' \cong \operatorname{End} E'$.

Task 3: Given two elliptic curves E, E', and the corresponding orders $\mathcal{O} \cong \operatorname{End} E$, $\mathcal{O}' \cong \operatorname{End} E'$, compute a connecting ideal I corresponding to an isogeny $\varphi_I\colon E \to E'$.

Task 4: Given a left ideal I of a maximal order $\mathcal{O} \subset \mathcal{B}_{p,\infty}$, find an equivalent ideal $J = I\beta$ (for $\beta \in \mathcal{B}^*_{p,\infty}$) with specific norm (usually small or smooth).

Task 5: Given $\mathcal{O} \cong \operatorname{End} E$, translate between isogenies $\varphi\colon E \to E'$ and their corresponding left $\mathcal{O}$-ideals I_φ.

Depending on the degree, Task 1 can be solved using Vélu's formulae [37] or the $\sqrt{\text{élu}}$ algorithm [5]. See Section 5.1 for more details. For Task 2 we can compute $\mathcal{O}'$ as $\mathcal{O}_R(I_\varphi)$ and the connecting ideal I in Task 3 satisfies $\mathcal{O} = \mathcal{O}_L(I)$, where the left order $\mathcal{O}_L(I)$ is defined analogously to the right order $\mathcal{O}_R(I) = \mathcal{O}'$. Task 4 is solved by the KLPT algorithm [28] and Task 5 is addressed by subroutines of SQISign [19]. The right orders of the equivalent ideals from Task 4 only agree up to isomorphism, and if we translate them with Task 5 the codomains of the resulting isogenies only agree up to Galois conjugacy.

Note that these are not very specific and might have significantly different running times for special cases or when given additional information. For example, Task 1 is more efficient for smooth degrees than for non-smooth ones of similar size (see Section 5.1) and Task 5 can be done faster when given extra information.

In general, Task 5 only requires corresponding generators $\mathcal{O} = \langle\alpha_0, \ldots, \alpha_3\rangle \cong \langle\alpha_0, \ldots, \alpha_3\rangle = \operatorname{End} E$. Given an ideal I, the kernel of the corresponding isogeny

φ_I is the set of points $K \in E$ such that $\alpha(K) = \infty$ for all $\alpha \in \operatorname{End} E$ corresponding to an element of I. Given an isogeny $\varphi\colon E \to E'$ with kernel $\langle K \rangle$, the corresponding ideal I_φ is the set of elements $\mathfrak{a} \in \mathcal{O}$ such that the corresponding $\alpha \in \operatorname{End} E$ satisfies $\alpha(K) = \infty$. Given a kernel generator K of order d and two special endomorphisms θ, η, Algorithm 23 from [13] shows that we can find a corresponding ideal efficiently. It computes $I_\varphi = \mathcal{O}\mathfrak{a} + \mathcal{O}d$ with $\mathfrak{a} = a + b\theta - \eta$ by solving $[a]K + [b]\theta(K) = \eta(K)$. If we know $K = [s]P + [t]Q \in E[d]$ for a torsion basis (P, Q) of $E[d]$, the action of θ and η can even be precomputed. The opposite direction can be done even more efficiently if we know the norm d and a generator $\mathfrak{a}$ of $I = \mathcal{O}\mathfrak{a} + \mathcal{O}d$ as well as a torsion basis (P, Q) of $E[d]$. In [24] they show that the corresponding isogeny has kernel $\langle \widehat{\alpha}(P), \widehat{\alpha}(Q) \rangle$ where the dual $\widehat{\alpha}$ corresponds to the conjugate $\overline{\mathfrak{a}}$.

To create a delay we need moderately hard problems, which are still polynomial in complexity but might take a considerable time to compute. In Section 5.1 we show that Task 1 can be made sufficiently slow. The following hard problems have a conjectured exponential complexity (see Section 5.1) and are equivalent [38]. They are the basis for encryption or signature schemes like CSIDH [11] or SQISign [19]. In our case they ensure that there are no shortcuts for the forced decommitment.

Problem 3.5 (Isogeny Path Problem)**.** *Given two (isogenous) supersingular elliptic curves E, E' and a prime ℓ, find a path from E to E' in the ℓ-isogeny graph.*

Problem 3.6 (Endomorphism Ring Problem)**.** *Given a supersingular elliptic curve E, find four endomorphisms that generate* $\operatorname{End} E$ *as a lattice.*

Problem 3.7 (Maximal Order Problem)**.** *Given a supersingular elliptic curve E, find four quaternions in $\mathcal{B}_{p,\infty}$ that generate a maximal order $\mathcal{O} \cong \operatorname{End} E$.*

Remark 3.8. *Knowledge of endomorphism rings can break the hard problems. If we know both endomorphism rings the first hard problem becomes polynomial using Tasks 3 - 5. If we know an isogeny from a curve with known endomorphism ring to our curve the third hard problem becomes polynomial by Task 2. Using Task 5 a solution for the third problem can be translated into a solution for the second hard problem.*

Finding supersingular elliptic curves can basically be done in two ways. We can reduce an elliptic curve in characteristic 0 modulo a prime and check if the resulting curve is supersingular, or take a random isogeny starting at one of these curves. In both cases the endomorphism ring of the final curve can be computed either via reduction or by transport along the isogeny. But as discussed in Remark 3.8 this weakens the hard problems. Hence, many cryptosystems require curves with unknown endomorphism ring. This in turn forces them to use a multi-party computation or a trusted authority in their setup to ensure that no single participant knows a complete path from a curve with known endomorphism ring to the one used. See [3] for more information. Note that the present cryptosystem has the advantage of not relying on a curve with unknown endomorphism ring.

4 The Protocol

Now we can combine the previous two sections and present our construction. First we give a high-level overview and discuss some challenges. Then we look at the algorithms and choices for the parameters.

4.1 Overview

At the heart of our protocol is an isogeny φ_T of degree d_T, which takes time t_{fd} to evaluate and hence causes the delay. Its domain is a public supersingular elliptic curve E_s with secret $\mathcal{O}_s \cong \operatorname{End} E_s$ and its kernel is generated by a publicly known point K_T on E_s. We use the j-invariant j_T of the codomain E_T of φ_T to hide the message $m \in M$. Therefore, an adversary needs to compute E_T (or rather $j_T = j(E_T)$) in order to break hiding or to open the commitment by force. We can choose how long the commitment should be kept secret by setting the degree d_T accordingly. This gives us hiding. Since E_s and K_T are part of the commitment, the codomain $E_T \cong E_s/\langle K_T \rangle$ is fixed (up to isomorphism) and we have perfect binding.

For verification to be faster than forced opening, we need a more efficient way to compute j_T. The starting curve E_0 has a known endomorphism ring, which allows us to compute elements of $\operatorname{End} E_0 \cong \mathcal{O}_0$ efficiently using precomputations. In the commitment we use `FullRepresentInteger` from [20] to find an $\gamma \in \operatorname{End} E_0$ of degree $d_s^2 d_T$ and decompose it into the two secret isogenies φ_s and ψ of smooth degree d_s and the public delay isogeny φ_T of degree d_T. This is visualized in Figure 1.

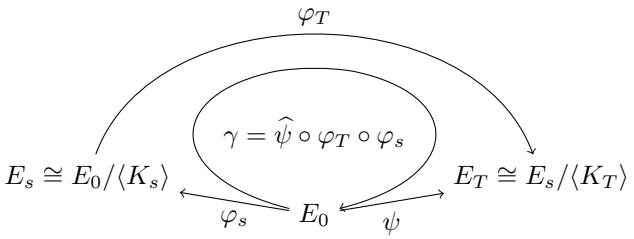

Figure 1: Decomposing $\gamma \in \operatorname{End} E_0$ of degree $d_s^2 d_T$ to find suitable isogenies.

We give γ to the verifier as part of the decommitment proof, so `Com` and `DecVrfy` can compute E_T as the codomain[4] of ψ. An adversary only knows E_s, but not φ_s and hence can neither compute $\mathcal{O}_s \cong \operatorname{End} E_s$ nor the ideals corresponding to φ_s or φ_T. Therefore, it has no efficient way to compute shortcuts. This gives us the preferred difference in speed between verification and forced opening.

To efficiently verify the validity of a commitment, we need to map the j-invariant j_T into the group of messages M. This map has to satisfy the following property. Otherwise the commitment might leak information about j_T.

Definition 4.1 (Inverse Resistant Functions)**.** *A function $f\colon X \to Y$ is* λ-inverse resistant*, if for all $y \in Y$ the preimage $f^{-1}(y) \subseteq X$ has at least 2^λ elements.*

[4] In practice, they only compute $\widetilde{E}_T$ isomorphic to E_T or its Galois conjugate E_T^p.

This definition is different from one-way functions, since finding an element in the preimage is allowed as long as the probability to find the original input is sufficiently small. It also differs from hash functions, which are mostly considered to be collision resistant. A simple projection with a sufficiently large preimage set satisfies this definition but is neither a one-way function nor a proper hash function.

4.2 Algorithms

As seen in Definition 2.1 we have five algorithms `PGen`, `Com`, `ComVrfy`, `DecVrfy` and `FDecom`. In this subsection we give pseudocode for each algorithm and discuss some subroutines. An implementation in SageMath can be found at https://github.com/kahrens-code/signitc. The code is based on the SageMath implementation of the "Learning to SQI" project[5].

4.2.1 Parameter Generation

The parameter generation `PGen` defines the security of the whole protocol and fixes the delay t_{fd}. It sets all general parameters like the characteristic p of the finite fields, the starting curve E_0, $\mathcal{O}_0 \cong \operatorname{End} E_0$, the degrees d_s and d_T, as well as the message group $(M, \oplus)$ and the inverse resistant function $F\colon J \to M$ where J is the set of all possible j-invariants of E_T. It also provides some precomputations that improve the speed of the commitment and the decommitment verification. These precomputations include bases of the d_s- and d_T-torsion groups of E_0. Its output is the common reference string **crs**. A detailed description can be found in Algorithm 1.

Algorithm 1 Parameter generation algorithm `PGen`

Require: Security parameter 1^κ, delay t_{fd}
Ensure: $\mathbf{crs} = (p, E_0, \operatorname{End} E_0, \mathcal{O}_0, M, F, d_s, P_s, Q_s, d_T, P_T, Q_T)$

1: Choose prime $p \geq 2^{2\kappa} - 1$
2: Choose supersingular elliptic curve E_0 with known $\mathcal{O}_0 \cong \operatorname{End} E_0$
3: Find corresponding bases $\mathcal{O}_0 = \langle \boldsymbol{\alpha}_0, \ldots, \boldsymbol{\alpha}_3 \rangle$ and $\operatorname{End} E_0 = \langle \alpha_0, \ldots, \alpha_3 \rangle$
4: Choose a group $(M, \oplus)$ with efficient membership testing as message space
5: Choose an efficient, inverse resistant function $F\colon J \to M$ oblivious to Galois conjugacy, i.e. $F(j) = F(j^p)$
6: Choose smooth $d_s \geq 2^{2\kappa}$ such that $E_0[d_s] \subseteq E_0(\mathbb{F}_{p^4})$
7: Choose $d_T(t_{\text{fd}}) \in \mathbb{N}$ such that d_T is coprime to d_s and $E_0[d_T] \subseteq E_0(\mathbb{F}_{p^4})$
8: Find torsion bases $\langle P_s, Q_s \rangle = E_0[d_s]$ and $\langle P_T, Q_T \rangle = E_0[d_T]$
9: **return crs** $= (p, E_0, \operatorname{End} E_0, \mathcal{O}_0, M, F, d_s, P_s, Q_s, d_T, P_T, Q_T)$

4.2.2 Commitment

The commitment algorithm `Com` takes as input a message $m \in M$ and outputs a tuple $(\mathbf{C}, \pi_{\text{com}}, \pi_{\text{dec}})$. The commitment itself $\mathbf{C} = (E_s, K_T, u)$ is again a tuple of a supersingular elliptic curve E_s, a point K_T on E_s that generates the kernel of φ_T and an element $u \in M$. While the commitment proof π_{com} is empty, the

[5] https://github.com/LearningToSQI/SQISign-SageMath

decommitment proof π_{dec} allows to use the same method for computing u as in the commitment algorithm.

The algorithm `FullRepresentInteger` actually returns $\gamma \in \mathcal{O}_0$ of norm $d_s^2 d_T$ instead of the corresponding $\gamma \in \text{End}\, E_0$. So the decomposition in Figure 1 is done in terms of ideals and then translated into kernel generators using the trick from [24]. We first compute ideals $I_s = \mathcal{O}_0\gamma + \mathcal{O}_0 d_s$, $I'_T = \mathcal{O}_0\gamma + \mathcal{O}_0 d_T$ and corresponding kernel generators $K_s, K'_T \in E_0$. The isogenies with these kernels are $\varphi_s \colon E_0 \to E_s$ and $\varphi'_T \colon E_0 \to E'_T$, respectively. Then the delay isogeny $\varphi_T \colon E_s \to E_T$ is the pushforward of φ'_T under φ_s. Note that $I_s \cap I'_T = I_{sT} = \mathcal{O}_0\gamma + \mathcal{O}_0 d_s d_T$ and this corresponds to $\varphi_T \circ \varphi_s$ and $\varphi'_s \circ \varphi'_T$ (see [19, Lemma 3]). So they form the SIDH-square of Figure 2. We only need to compute $E_s \cong E_0/\langle K_s \rangle$ as codomain of φ_s and the point $K_T = \varphi_s(K'_T) \in E_s$ generating the kernel of φ_T.

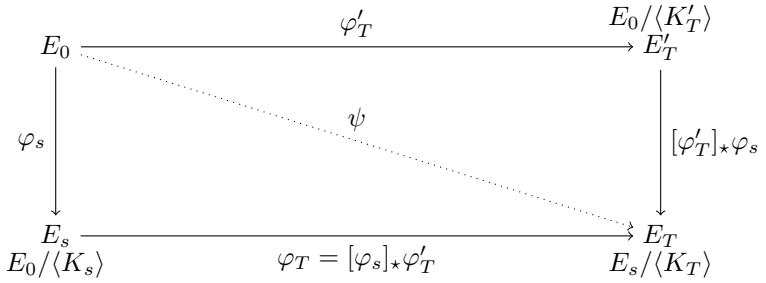

Figure 2: Walk in the isogeny graph with fast secret isogeny φ_s, slow delay isogeny φ_T and fast shortcut isogeny ψ, where $K_T = \varphi_s(K'_T)$.

The element $u = m \ominus F(j_T) \in M$ is the message m masked[6] by the j-invariant j_T of $E_T \cong E_s/\langle K_T \rangle$. To compute this efficiently we use the shortcut isogeny $\psi \colon E_0 \to \widetilde{E}_T$. So we compute $I_\psi = \mathcal{O}_0\overline{\gamma} + \mathcal{O}_0 d_s$, a point K_ψ generating the corresponding kernel and then $\widetilde{E}_T \cong E_0/\langle K_\psi \rangle$ as codomain of ψ. Since I_{sT} and I_ψ are equivalent ideals we can find a $\beta \in \mathcal{B}_{p,\infty}$ such that $I_{sT}\beta = I_\psi$ and their right orders agree up to isomorphism. Hence, the codomains E_T of φ_T (or rather $\varphi_T \circ \varphi_s$) and $\widetilde{E}_T$ of ψ have to agree up to Galois conjugacy. Let j_T and $\widetilde{j}_T$ be the j-invariants of E_T and $\widetilde{E}_T$, respectively. Then we have $F(j_T) = F(\widetilde{j}_T)$ as we chose F oblivious to Galois conjugacy. This allow us to compute u as $m \ominus F(\widetilde{j}_T)$. For a fast verification the elements $\gamma \in \mathcal{O}_0$ and $\beta \in \mathcal{B}_{p,\infty}$ are given in the decommitment proof $\pi_{\text{dec}} = (\gamma, \beta)$. This allows the verifier to also use the shortcut isogeny ψ. The individual steps are given in Algorithm 2.

Remark 4.2. *Since translating from ideals to kernel points is faster than the other direction (in our case), we use γ in π_{dec} instead of K_s and K'_T. Computing shortcut isogenies with KLPT [28] or using higher dimensional representations (e.g. [4, 16, 29, 30]) is more involved and requires special primes. In the case of KLPT it even yields worse results as smooth outputs are of size roughly p^3 [31]. Many higher dimensional approaches also use* `FullRepresentInteger`*, so they are probably slower than our simple solution.*

[6]Here $\ominus$ means the addition of the inverse in the group $(M, \oplus)$ such that $m \ominus m$ is the neutral element in M and in particular $u \oplus F(j) = m \ominus F(j) \oplus F(j) = m$.

Algorithm 2 Commitment algorithm `Com`

Require: Common reference string **crs**, message $m \in M$
Ensure: $(\mathbf{C}, \pi_{\text{com}}, \pi_{\text{dec}}) = \big((E_s, K_T, u), (), (\gamma, \beta)\big)$

Use `FullRepresentInteger` to find (random) $\gamma \in \mathcal{O}_0$ of norm $d_s^2 d_T$
Compute ideals $I_{sT} = \mathcal{O}_0\gamma + \mathcal{O}_0 d_s d_T$ and $I_\psi = \mathcal{O}_0\overline{\gamma} + \mathcal{O}_0 d_s$
Compute $\beta \in \mathcal{O}_0$ such that $I_{sT}\beta = I_\psi$

Compute ideal $I_s = \mathcal{O}_0\gamma + \mathcal{O}_0 d_s$
Compute a corresponding kernel point K_s and isogeny $\varphi_s \colon E_0 \to E_s$
Compute $E_s \cong E_0/\langle K_s \rangle$ via Vélu's formulae
Compute ideal $I'_T = \mathcal{O}_0\gamma + \mathcal{O}_0 d_T$ and a corresponding kernel point K'_T
Compute $K_T = \varphi_s(K'_T) \in E_s[d_T]$

Compute a kernel point K_ψ and isogeny $\psi \colon E_0 \to \widetilde{E}_T$ corresponding to I_ψ
Compute $\widetilde{E}_T \cong E_0/\langle K_\psi \rangle$ via Vélu's formulae
Compute $\widetilde{j}_T = j(\widetilde{E}_T)$ and $u = m \ominus F(\widetilde{j}_T) \in M$ ▷ $F(\widetilde{j}_T) = F(j(E_T))$

Set $\mathbf{C} = (E_s, K_T, u)$ ▷ *Commitment*
Set $\pi_{\text{com}} = ()$ ▷ *Commitment proof (empty)*
Set $\pi_{\text{dec}} = (\gamma, \beta)$ ▷ *Decommitment proof*
return $(\mathbf{C}, \pi_{\text{com}}, \pi_{\text{dec}})$

4.2.3 Commitment Verification

Algorithm 3 shows the commitment verification `ComVrfy`. It is fast since it only needs to check if the three parts of the commitment are of the correct form. Namely, E_s is an elliptic curve, K_T is a point of order d_T on that curve and u is an element of the group M.

Algorithm 3 Commitment verification algorithm `ComVrfy`

Require: Common reference string **crs**, commitment $\mathbf{C}$ and proof π_{com}

Check if E_s is an elliptic curve over $\mathbb{F}_{p^4}$
Check if K_T is a point on E_s with $\operatorname{ord} K_T = d_T$
Check if $u \in M$
return (**accept**/**reject**)

4.2.4 Decommitment Verification

The decommitment verification `DecVrfy` (Algorithm 4) is similar to the commitment algorithm. It first reconstructs I_s, I'_T from γ and computes corresponding kernel generators K_s, K'_T, respectively. Then it verifies that the codomain of isogeny $\varphi_s \colon E_0 \to E_0/\langle K_s \rangle$ is isomorphic to E_s. The kernel generators are not unique, but they have to be multiples of each other. To check if K_T and $\varphi_s(K'_T)$ generate the same subgroup, it could compute a discrete logarithm to check if one is the multiple of the other. Since it does not need the factor, a faster way is to compute the Weil pairing and check if $e(K_T, \varphi_s(K'_T)) = 1$.

This makes sure that the ideals I_s and I'_T agree with the commitment and the verifier can construct the shortcut from these verified components. The

algorithm computes the ideal $I_{sT} = I_s \cap I'_T$ corresponding to $\varphi_T \circ \varphi_s$ and the equivalent ideal $I_\psi = I_{sT}\beta$. It first verifies that I_ψ has norm d_s and is generated by γ. Then it uses this information to compute a point K_ψ generating the kernel of the corresponding isogeny $\psi\colon E_0 \to E_0/\langle K_\psi\rangle$. Finally, it computes $\widetilde{E}_T \cong E_0/\langle K_\psi\rangle$, its j-invariant $\widetilde{j}_T$ and checks if $u \oplus F(\widetilde{j}_T) = m$. This gives the correct result since $F(j_T) = F(\widetilde{j}_T)$.

Algorithm 4 Decommitment verification algorithm `DecVrfy`

Require: Common reference string **crs**, commitment **C**
Require: Message m, decommitment proof π_{dec}
1: Compute ideal $I_s = \mathcal{O}_0\gamma + \mathcal{O}_0 d_s$
2: Compute a corresponding kernel point K_s and isogeny $\varphi_s\colon E_0 \to E_0/\langle K_s\rangle$
3: Check if $E_0/\langle K_s\rangle \cong E_s$
4: Compute ideal $I'_T = \mathcal{O}_0\gamma + \mathcal{O}_0 d_T$ and a corresponding kernel point K'_T
5: Compute $\varphi_s(K'_T) \in E_s[d_T]$ and check if $e(\varphi_s(K'_T), K_T) = 1$

6: Compute ideal $I_{sT} = I_s \cap I'_T$ and equivalent ideal $I_\psi = I_{sT}\beta$
7: Check if I_ψ has norm d_s and is generated by $\mathcal{O}_0\overline{\gamma} + \mathcal{O}_0 d_s$
8: Compute a corresponding kernel point K_ψ and isogeny $\psi\colon E_0 \to E_0/\langle K_\psi\rangle$
9: Compute $\widetilde{E}_T \cong E_0/\langle K_\psi\rangle$ and $\widetilde{j}_T = j(\widetilde{E}_T)$
10: Check if $u \oplus F(\widetilde{j}_T) = m$ ▷ $F(\widetilde{j}_T) = F(j(E_T))$
11: **return** (**accept**/**reject**)

4.2.5 Forced Decommitment

In terms of the number of tasks the forced decommitment algorithm is rather simple. It first checks if the input is as expected. Then it just computes E_T as codomain of the isogeny φ_T given by the point K_T that generates its kernel. From there it recovers the message $m = u \oplus F(j(E_T))$.

Algorithm 5 Forced decommitment algorithm `FDecom`

Require: Common reference string **crs**, commitment **C**
Ensure: Message m or **reject**
1: If $\texttt{ComVrfy}(\textbf{crs}, C) = \textbf{reject}$ then **return reject**
2: Compute $E_T \cong E_s/\langle K_T\rangle$ via Vélu's formulae or $\sqrt{\text{élu}}$ algorithm
3: Compute $j_T = j(E_T)$ and $m = u \oplus F(j_T)$
4: **return** m

4.3 Parameter Sizes and other Choices

The algorithms above do not specify all properties of the parameters. Therefore, we now discuss the necessary and some optional choices. For example, the hiding property sets requirements on the size of some parameters and we also propose some choices for implementing this protocol. In Section 6 we give an example how explicit numbers may look like.

The delay t_{fd} should be large, but it has to be polynomial in κ. On one hand the main idea of NITC schemes is that we can forcefully open a commitment (in

polynomial time) with FDecom, if someone refuses to open it themselves. On the other hand generic algorithms to solve Problems 3.5 - 3.7 could be faster than FDecom and therefore violate hiding, if t_{fd} was exponential in κ. In particular, we need $t_{\text{fd}} < 2^{\kappa/2}$ due to Assumptions 5.5 and 5.6 of Section 5.1 for quantum security and $t_{\text{fd}} \gg t_{\text{com}}, t_{\text{cv}}, t_{\text{dv}}$.

4.3.1 Prime p, Starting Curve E_0 and Isogenies φ_s and φ_T

The starting curve could be any supersingular elliptic curve E_0 with a known efficient representation of $\mathcal{O}_0$. For our protocol we choose E_0 to be the curve $E_0 \colon y^2 = x^3 + x$ with $(p+1)^2$ points over $\mathbb{F}_{p^2}$. To have $\mathcal{O}_0 = \langle 1, \mathrm{i}, \frac{\mathrm{i}+\mathrm{j}}{2}, \frac{1+\mathrm{k}}{2} \rangle_{\mathbb{Z}}$ we choose $p \equiv 3 \bmod 4$. In this case the endomorphisms $[\mathrm{i}] \colon (x, y) \mapsto (-x, \mathrm{i}y)$ and $\phi \colon (x, y) \mapsto (x^p, y^p)$ (Frobenius map) correspond to i and j, respectively. This also allows for more efficient translation from ideals to kernel points. In order to satisfy the hiding property, p and d_s have to have a certain size. It has to be infeasible to precompute $\mathcal{O}_s$ for all possible E_s or to find an isogeny from E_0 to E_s in time less than t_{fd} in the online phase. Therefore, we choose $p \geq 2^{2\kappa} - 1$, $d_s \geq 2^{2\kappa}$ and $p \approx d_s$. In Section 5 we give a more detailed justification of these numbers.

Usually, we would want d_s and d_T to be smooth numbers both dividing $p+1$ in order to have fast evaluation of the corresponding isogenies. So d_s should be smooth and divide $p+1$ (or p^2-1). However, evaluating φ_T does not need to (in fact should not) be fast, since it is only evaluated by FDecom. Therefore, d_T can contain larger prime factors and does not need to divide $p+1$ (or p^2-1). It is chosen such that computing an isogeny of degree d_T takes at least time t_{fd} (and preferably not much longer). Note that we can choose d_T as a large composite number with many prime factors to have more confidence in sequentiality. Since $K_T = \varphi_s(K'_T)$ has to be of order d_T and for the decomposition of the ideals, we need the degree d_s of φ_s to be coprime to d_T.

For a supersingular curve with $(p+1)^2$ points over $\mathbb{F}_{p^2}$ we have $(p^e - (-1)^e)^2$ points over $\mathbb{F}_{p^{2e}}$ and the largest fully $\mathbb{F}_{p^{2e}}$-rational torsion group is the $(p^e - (-1)^e)$-torsion. If d_T is large or contains prime factors that do not divide $p+1$, this means that we need to go to extensions of $\mathbb{F}_{p^2}$ to find a basis for the d_T-torsion group of E_0 or E_s. Higher extensions and larger p slow down the computations, therefore we want to minimize the degree of the extension and the size of p to increase efficiency. The size of p affects almost all computations, whereas the size of e only influences computations related to the K_T or φ_T. Both Com and DecVrfy have to compute K_T, but not φ_T. Thus, for longer delays it can be beneficial to make d_T less smooth rather than making it larger. This can increase the delay without increasing the extension degree e, i.e. without slowing down Com and DecVrfy.

For our implementation we choose d_T to be prime and p such that $e = 2$ is sufficient to find a factor d_T in $p^e - (-1)^e$. This ensures fast commitment and verification. We allow at most $O(t_{\text{fd}})$ parallel processes for an adversary for our security proofs. Finding a suitable size for d_T with respect to t_{fd} can be done using the formula $d_T \gtrsim t_{\text{fd}}^{3.52}$ from Assumption 5.8. The primes used in other isogeny-based cryptosystems allow to choose d_s this way. The only difficulty is to find ones that have suitably sized prime factors d_T in $p^2 - 1$.

Remark 4.3. *We could also allow more parallelism in exchange for larger d_T. In particular, choosing d_T to be smooth allows for up to* $\text{poly}(t_{\text{fd}})$ *cores. The*

downside is that we need larger extension fields and proving an asymptotic gap between verification and forced decommitment is harder. It might be necessary to choose t_{fd} subexponential in κ in this case.

4.3.2 Message Space M and Function F

We choose M to be a finite group $M = \mathbb{Z}/N\mathbb{Z}$ for an integer $N \in \mathbb{N}$. This gives us very efficient membership testing and group operations. The size of N depends on the needed length of a message m and the security parameter κ. The set J of all possible j-invariants of E_T is a subset of all j-invariants and hence $|J| \leq \lfloor p/12 \rfloor + 2$. Since E_T is isomorphic to $\widetilde{E}_T$ or $\widetilde{E}_T^p$, the size of J is also bound by the number of isogenies of degree d_s starting at E_0. We can approximate this number as $d_s \geq 2^{2\kappa}$. Since $p \approx d_s$ we can assume that we can reach every j-invariant. If N is larger than $|J|$, then $F\colon J \to M$ can not be surjective and therefore $u = m \ominus F(j_T)$ might leak information about the message m.

As mentioned before, computing j_T from $F(j_T)$ has to be infeasible or at least slow. In order to satisfy hiding we choose the function F to be λ-inverse resistant with $\lambda = \frac{3}{2}\kappa$. In addition, it has to be fast since `Com` and `DecVrfy` have to compute $F(j_T)$. An easy way to accomplish this is to take a function that is not injective. The larger the kernel of F, i.e. smaller N, the more information is lost. A simple projection $\mathbb{F}_{p^2} \supset J \to \mathbb{F}_p$ onto one of the components or even their sum will leak information, since there is a subset of j-invariants that already are in $\mathbb{F}_p$. If we use a simple map like $(a, b) \mapsto b \bmod N$ or $(a, b) \mapsto a + b \bmod N$, we thus need to use $N \ll p$.

The right orders of equivalent ideals are only isomorphic and the corresponding shortcut isogenies have codomains with j-invariants that agree only up to Galois conjugacy. It is easier to choose F oblivious to Galois conjugacy than to ensure equality of orders or j-invariants. For an implementation we can identify $J \subset \mathbb{F}_{p^2}$ with a subset of $\mathbb{F}_p[\mathrm{i}] \cong \mathbb{F}_{p^2}$ and choose

$$F\colon J \to M = \mathbb{Z}/N\mathbb{Z}, \quad a + b\mathrm{i} \mapsto a + |b| \bmod N.$$

Since we choose $p \equiv 3 \bmod 4$ we have $(a+b\mathrm{i})^p = a - b\mathrm{i}$ and $F(j) = F(j^p)$. Also, F should be $\frac{3}{2}\kappa$-inverse resistant, so we take $N \leq \lfloor 2^{\kappa/2}/12 \rfloor \leq \lfloor p^{1/4}/12 \rfloor$. Then we can expect every element in M to be the image of about $p^{3/4} \gtrsim 2^{3\kappa/2}$ elements in J. There is no direct way of finding the supersingular j-invariants. Hence, one would have to compute the preimage in $\mathbb{F}_{p^2}$ (about $12p^{7/4}$ elements) and check if they are j-invariants of supersingular elliptic curves. This is sufficiently inverse resistant in practice.

Remark 4.4 (Publicly Verifiable)**.** *To make the scheme publicly verifiable, we can add an encryption* $\mathrm{Enc}_{j_T}(\pi_{\text{dec}})$ *of* π_{dec} *to the commitment such that* j_T *is the key for the decryption* $\mathrm{Dec}_{j_T}(\mathrm{Enc}_{j_T}(\pi_{\text{dec}})) = \pi_{\text{dec}}$*. This allows* `FDecom` *to provide the decommitment proof* π_{dec} *and everyone could use* `DecVrfy` *to verify the output of* `FDecom` *instead of computing it themselves.*

5 Security

We show that our protocol satisfies the Definition 2.1 of a NITC scheme by Katz et al. [27] and prove the three properties practicality, hiding and binding. These

proofs are based on assumptions for the relative speed of some algorithms.

Our algorithms have the correct input and output arguments, and for all κ and $m \in M$ every set of honestly generated $(\kappa, m, \mathbf{crs}, \mathbf{C}, \pi_{\text{com}}, \pi_{\text{dec}})$ satisfies verification $\texttt{ComVrfy}(\mathbf{crs}, \mathbf{C}, \pi_{\text{com}}) = \mathbf{accept} = \texttt{DecVrfy}(\mathbf{crs}, \mathbf{C}, m, \pi_{\text{dec}})$ and forced decommitment $\texttt{FDecom}(\mathbf{crs}, \mathbf{C}) = m$. This makes it a NITC scheme.

5.1 Relative Running Times

Before we can tackle practicality or hiding, we have to look at the time it takes to find and to evaluate isogenies. Note that our timings are the number of (field) operations rather than real-world times. In Section 5.1.1 we present the total number of operations required for several tasks. This corresponds to sequential time, i.e. computing everything without parallelization. In Section 5.1.2 we then discuss how these tasks can be parallelized.

5.1.1 Sequential Time

Computing isogenies of prime degree q can be done using Vélu's formulae in time $O(q)$, or the $\sqrt{\text{élu}}$ algorithm [5] in time $\sqrt{q}(\log q)^{2+o(1)}$ or $\widetilde{O}(\sqrt{q})$ for short[7]. The crossover point for optimized algorithms is at $q \approx 100$, and we denote the (sequential) time it takes to compute an isogeny of prime degree q with $\text{eval}_{\text{prime}}(q)$.

Remark 5.1. *If the kernel* $\ker\varphi$ *of an isogeny* $\varphi\colon E \to E'$ *or a point* $P \in E$ *in its domain is only* $\mathbb{F}_{p^{2e}}$*-rational, we need the same number of operations to find the codomain* $E/\ker\varphi$ *or to evaluate* $\varphi(P)$*, but they could be operations in* $\mathbb{F}_{p^{2e}}$ *instead of* $\mathbb{F}_{p^2}$*. The point* K_T *of the commitment might only be defined over extension fields and we need to compute* $\varphi_s(K'_T) = K_T$*. Therefore, we only considered the number of operations and do not distinguish between operations in* $\mathbb{F}_{p^2}$ *and more costly operations in extension fields* $\mathbb{F}_{p^{2e}}$*. The majority of operations of* `FDecom` *may be in extension fields, but for* `Com`*,* `ComVrfy` *and* `DecVrfy` *most operations can be done in* $\mathbb{F}_{p^2}$*. So our timings are rather conservative.*

Lemma 5.2. *Given an elliptic curve* E *and a point* $K \in E$ *of prime order* q*, there is a (small) constant* c_p *such that evaluating the corresponding isogeny* φ_K *with kernel* $\ker\varphi_K = \langle K\rangle$ *at any point on* E *or computing the codomain* $E/\langle K\rangle$ *takes sequential time* $\text{eval}_{\text{prime}}(q) \le c_p q$*.*

Now let us look at an isogeny φ with a kernel that is generated by a point K'_0 of order q^k. We can decompose $\varphi = \varphi_k \circ \cdots \circ \varphi_1$ into isogenies φ_i of degree q. In each step we compute the points $K_i = [q^{k-i}]K'_{i-1}$ generating the kernel of φ_i and $K'_i = \varphi_i(K'_{i-1})$ generating the kernel of $\varphi'_i = \varphi_k \circ \cdots \circ \varphi_{i+1}$. So every step takes time $\text{eval}_{\text{prime}}(q)$ plus the time it takes to compute the point multiplication. Generalizing this to isogenies of arbitrary composite degree gives us bounds for the time $\text{eval}(d)$ it takes to compute an isogeny of degree d.

Lemma 5.3. *Given an elliptic curve* E *and a point* $K \in E$ *of order* d *with prime factorization* $d = \prod_{i=1}^{r} q_i^{e_i}$*. There is a (small) constant* $c_c \ge 1$ *such that the sequential time* $\text{eval}(d)$ *it takes to evaluating the corresponding isogeny* φ_K

[7]Here $\widetilde{O}$ may also contain additional logarithmic terms $\widetilde{O}(n) = O(n\,\text{poly}(\log n))$.

with kernel $\ker \varphi_K = \langle K \rangle$ *at any point on* E *or computing the codomain* $E/\langle K \rangle$ *is bounded by*

$$\sum_{i=1}^{r} e_i \operatorname{eval}_{\text{prime}}(q_i) < \operatorname{eval}(d) < c_c \sum_{i=1}^{r} e_i (\operatorname{eval}_{\text{prime}}(q_i) + \log d).$$

Proof. Let $n = \sum_{i=1}^{r} e_i$ and $\varphi = \varphi_n \circ \cdots \circ \varphi_1$. For each isogeny φ_j in the chain we need to compute the kernel of the current isogeny via a multiplication by $d/(\prod_{k \leq j} \deg \varphi_k) < d$, its codomain and the kernel of the remaining isogeny. Each multiplication takes time less than $c_c \log d$ and for the lower bound we ignore the computation of the kernel points. □

Combining the previous two lemmas we get an upper bound for the computation time of isogenies of smooth degree.

Lemma 5.4. *An isogeny of degree* d *with prime factorization* $d = \prod_{i=1}^{r} q_i^{e_i}$ *and* $q_i < B$ *(*B*-smooth) can be evaluated in time* $O(\frac{B}{\log B} \log d, (\log d)^2)$.

Proof. We use Lemmas 5.3 and 5.2 to write

$$\operatorname{eval}(d) < c_c \sum_{i=1}^{r} e_i (\operatorname{eval}_{\text{prime}}(q_i) + \log d) \leq c_c \sum_{i=1}^{r} e_i (c_p q_i + \log d).$$

Since $q_i < B$ for all $1 \leq i \leq r$, we get $q_i \leq \log q_i \frac{B}{\log B}$ and with the upper bound $\sum_{i=1}^{r} e_i \leq \sum_{i=1}^{r} e_i \log q_i = \log d$ we can write

$$\operatorname{eval}(d) < c_c c_p \sum_{i=1}^{r} e_i \log q_i \frac{B}{\log B} + c_c \sum_{i=1}^{r} e_i \log d \leq c_c c_p \frac{B}{\log B} \log d + c_c (\log d)^2. \ \square$$

According to Eisenträger et al. [23] the fastest (currently known) algorithms for solving the (equivalent) general Isogeny Path Problem, general Endomorphism Ring Problem or general Maximal Order Problem (cf. Section 3.2) over $\mathbb{F}_{p^2}$ take time $\widetilde{O}(p^{1/2})$ for classical computations and $\widetilde{O}(p^{1/4})$ with a quantum computer. Since E_0 and E_s are known to be connected by a d_s-isogeny there is also a meet-in-the-middle or claw-finding attack in classical time $\widetilde{O}(d_s^{1/2})$ and $\widetilde{O}(d_s^{1/4})$ when applying Grover's Algorithm [25].

Assumption 5.5 (General Isogeny Assumption)**.** *We assume that the fastest algorithms to solve the general Isogeny Path Problem, the general Endomorphism Ring Problem or the general Maximal Order Problem over* $\mathbb{F}_{p^2}$ *need at least* $p^{1/2}$ *or* $p^{1/4}$ *operations for classical or quantum algorithms, respectively.*

Assumption 5.6 (Special Isogeny Assumption)**.** *We assume that the fastest algorithms to find an isogeny between two* d*-isogenous curves over* $\mathbb{F}_{p^2}$ *take at least* $\min\{d^{1/2}, p^{1/2}\}$ *or* $\min\{d^{1/4}, p^{1/4}\}$ *operations for classical or quantum algorithms, respectively.*

5.1.2 Parallel Time

As we have seen at the beginning of Section 5.1.1, isogenies of prime degree q can be computed using Vélu or $\sqrt{\text{élu}}$. Vélu's formulae can be parallelized

perfectly to parallel time $O(q/n)$ for $n < q$ cores and $O(\log q)$ for $n = q$ cores. According to [15] parallelization of the $\sqrt{\text{élu}}$ algorithm only seems to be possible with diminishing returns on the number of cores. They give a complex formula for the asymptotic advantage, but we can bound it by a factor of $5n^{0.378}$ for n cores. So we can achieve parallel time $\widetilde{O}(\sqrt{q}/n^{0.378})$.

When computing an isogeny of smooth (composite) degree d, Lemma 5.4 shows that it can be done in sequential time $O((\log d)^2)$. To our knowledge, only the computations within each step can be parallelized, but the different steps along the chain have to be computed sequentially. So for long chains of isogenies of small degree the time it takes to compute it is already dominated by the number of steps and parallelization gives no significant advantage.

Assumption 5.7. *Let $d = \prod_{i=1}^{r} q_i^{e_i}$ be the prime factorization of the smooth degree d. Given an elliptic curve E and a point $K \in E$ of order d, we assume that evaluating the corresponding isogeny φ_K with kernel $\ker \varphi_K = \langle K \rangle$ at any point on E or computing the codomain $E/\langle K \rangle$ takes at least parallel time $\sum_{i=1}^{r} e_i$, independently from the number of cores.*

For isogenies of (larger) prime degree q we have to consider different levels of allowed cores. Given unbounded parallelism we can always compute the isogeny in parallel time $O(\log q)$. But as mentioned at the end of Section 2, this is not useful. We want to set the allowed number of cores in terms of the targeted delay. Let t be the delay we expect to achieve with an isogeny of prime degree q, i.e. a (close) lower bound for the parallel time it takes to compute the isogeny. If we would allow $\text{poly}(t)$ cores, we would get a paradox: When choosing $t = \log q$ we can not use q cores since q is not in $\text{poly}(t)$ and the fastest option to compute the isogeny takes time $\widetilde{O}(\sqrt{q})$ using $\sqrt{\text{élu}}$. So we should choose $t = q^r$ for some number $0 < r < 0.5$, but then we can use $t^{\lceil 1/r \rceil} \geq q$ cores and hence compute the isogeny in time $O(\log q)$ using fully parallelized Vélu's formulae. At least asymptotically this is much faster than $t = q^r$. This would again suggest choosing $t = \log q$.

This problem can be solved by allowing only $O(t^n)$ cores for a fixed $n \in \mathbb{N}$. Asymptotically t^{n+1} is larger than any $ct^n \in O(t^n)$ and we can set

$$t \approx \frac{\sqrt{q}}{(t^{n+1})^{0.378}}.$$

For our protocol we choose $n = 1$, i.e. $O(t)$ allowed cores, and $t = q^{0.284}$. When using t^2 cores the parallel time to compute the isogeny is $O(q^{0.432})$ with Vélu's formulae and $\widetilde{O}(q^{0.285})$ with $\sqrt{\text{élu}}$. If we assume that Vélu's formulae and the $\sqrt{\text{élu}}$ algorithm are close to optimal in computing prime degree isogenies we get the following assumption:

Assumption 5.8. *Given an elliptic curve E and a point $K \in E$ of prime order q, we assume that evaluating the corresponding isogeny φ_K with kernel $\ker \varphi_K = \langle K \rangle$ at any point on E or computing the codomain $E/\langle K \rangle$ takes at least parallel time $q^{0.284}$ when using $O(q^{0.284})$ cores.*

Assumptions 5.5 and 5.6 still hold when using $O(t)$ cores as long as $t \in \text{poly}(\log d, \log p)$. Now we can prove that computing the codomain of an isogeny can be made almost arbitrarily slow.

Theorem 5.9. *Let E be a supersingular elliptic curve over $\mathbb{F}_{p^2}$ with unknown $\mathcal{O} \cong \operatorname{End} E$, but d'-isogenous to a curve E_0 with known endomorphism ring. Fix a delay t such that $t \in \operatorname{poly}(\log d, \log p)$ and especially $t < \min\{d'^{1/4}, p^{1/4}\}$. Let K be a point on E of order d, such that computing the corresponding isogeny on $O(t)$ cores takes at least parallel time t, according to Assumption 5.8. Under Assumptions 5.5 and 5.6 computing $E_K \cong E/\langle K \rangle$ on $O(t)$ cores then takes at least time t.*

Proof. The isogeny $\varphi\colon E \to E_K$ with kernel $\langle K \rangle$ has degree d. Efficiently calculating a shortcut isogeny $\widetilde{\varphi}\colon E \to E_K$ or an efficient higher dimensional representation requires knowledge of $\mathcal{O} \cong \operatorname{End} E$. Finding the endomorphism ring $\operatorname{End} E$ or the order $\mathcal{O} \cong \operatorname{End} E$ without an isogeny $\varphi'\colon E_0 \to E$, or finding an isogeny $\widetilde{\varphi}$ without $\mathcal{O} \cong \operatorname{End} E$ are hard problems. By Assumption 5.5 finding $\operatorname{End} E$ or $\mathcal{O}$ takes time at least $p^{1/4} > t$. Finding an isogeny φ' needs at least time $\min\{d'^{1/4}, p^{1/4}\} > t$ by Assumption 5.6. Therefore, computing $E_K \cong E/\langle K \rangle$ on $O(t)$ cores takes at least time t by Assumption 5.8. □

The algorithm `FDecom` only has **crs** and $\mathbf{C} = (E_s, K_T, u)$ as input. In order to compute $m = u \oplus F(j_T)$ it has to calculate the j-invariant j_T of the secret curve $E_T \cong E_s/\langle K_T \rangle$. Theorem 5.9 gives us the following corollary:

Corollary 5.10. *For $t_{\mathrm{fd}} \in \operatorname{poly}(\kappa)$, $t_{\mathrm{fd}} < 2^{\kappa/2}$ and under the Assumptions 5.5, 5.6 and 5.8, the forced decommitment* `FDecom` *takes at least time t_{fd} when using $O(t_{\mathrm{fd}})$ cores.*

Note that the restriction $t_{\mathrm{fd}} < 2^{\kappa/2}$ is based on the quantum timings in Assumptions 5.5 and 5.6. For classical algorithms $t_{\mathrm{fd}} < 2^{\kappa}$ would be sufficient, but since our protocol should be quantum secure we chose the more general bound including quantum algorithms.

Remark 5.11. *For a delay isogeny of smooth degree we get a similar result with $\operatorname{poly}(t)$ allowed cores under Assumption 5.7 instead of 5.8.*

5.2 Hiding and Binding

For hiding we use the same (non-malleability) Definition 2.4 as Katz et al. [27]. First we show why we need an adapted security game. In Definition 2.3 the adversary $\mathcal{A}$ sends two messages m_0, m_1 and receives the commitment $\mathbf{C}_b = (E_s, K_T, u_b)$ corresponding to message m_b for a uniform $b \in \{0, 1\}$. It is allowed to query an oracle for $\texttt{FDecom}(\cdot)$ except for $\texttt{FDecom}(\mathbf{crs}, \mathbf{C}_b)$.

Lemma 5.12. *An adversary $\mathcal{A}$ can break hiding with the original security game from Definition 2.3.*

Proof. Since $m_{1-b} \ominus m_b \oplus u_b = u_{1-b}$, querying $\texttt{FDecom}(\mathbf{crs}, (E_s, K_T, u_\pm))$ with $u_+ = (m_0 \ominus m_1) \oplus u_b$ and $u_- = \ominus(m_0 \ominus m_1) \oplus u_b$ gives m_{1-b} and a random message m'. For $|M| = 2$ we have $u_+ = u_-$ and get m_{1-b}. For $|M| > 2$ however, we can assume $m_0 \neq m' \neq m_1$. This allows $\mathcal{A}$ to output the correct $b' = b$ with high probability.

Even worse, if we replace K_T by any other point K' such that $\langle K' \rangle = \langle K_T \rangle$, e.g. $K' = [\ell]K_T$ for ℓ coprime to d_T, or apply an isomorphism such that $E'/\langle K' \rangle \cong E_T \cong E_s/\langle K_T \rangle$ then $\texttt{FDecom}(\mathbf{crs}, (E', K', u_b))$ will return m_b. □

Thus, it is reasonable to disallow queries of the form $\texttt{FDecom}(\mathbf{crs}, (E', K', \cdot))$ for $E'/\langle K' \rangle \cong E_T \cong E_s/\langle K_T \rangle$. Since F is oblivious to Galois conjugacy, we also disallow queries with $E'/\langle K' \rangle \cong E_T^p$ and use the adapted security game in Definition 5.13.

Definition 5.13 (IND-CCA adapted)**.** *For a NITC scheme* TC *and an algorithm* $\mathcal{A}$*, define the game* $\texttt{IND-CCA}^{\mathcal{A}}_{\texttt{TC}}$ *as follows:*

1. *Compute* $\mathbf{crs} \leftarrow \texttt{PGen}(1^\kappa)$.
2. *Run* $\mathcal{A}(\mathbf{crs})$ *in a preprocessing phase with access to* $\texttt{FDecom}(\mathbf{crs}, \mathbf{C})$.
3. *When* $\mathcal{A}$ *outputs* (m_0, m_1)*, choose a uniform bit* $b \leftarrow \{0, 1\}$ *and then compute* $(\mathbf{C}_b, \pi_{\text{com}}, \pi_{\text{dec}}) \leftarrow \texttt{Com}(\mathbf{crs}, m_b)$*. Give* $(\mathbf{C}_b, \pi_{\text{com}})$ *to* $\mathcal{A}$*, who continues to have access to* $\texttt{FDecom}(\mathbf{crs}, \mathbf{C})$ *except that it may not query the oracle on* $(E', K', \cdot)$ *for* $E'/\langle K' \rangle$ *isomorphic to* $E_s/\langle K_T \rangle$ *or its Galois conjugate where* $\mathbf{C}_b = (E_s, K_T, u_b)$.
4. *When* $\mathcal{A}$ *outputs a bit* b'*, it wins iff* $b' = b$.

This is still in the spirit of the original definition, since it prohibits the "decryption" of the commitment in question. In our case the security arises from the secret isogeny $\varphi_s \colon E_0 \to E_s$ and the delay isogeny $\varphi_T \colon E_s \to E_T$ with kernel $\langle K_T \rangle$, and the "key" is $F(j_T)$ for $j_T = j(E_T)$. Such queries would enable $\mathcal{A}$ to find $F(j_T)$ and would hence basically allow to query $\texttt{FDecom}(\mathbf{crs}, (E_s, K_T, u_b))$ by proxy, which is forbidden in the original definition.

Assumption 5.14. *Let* φ_s*,* φ_T *and* ψ *be isogenies of degrees* d_s*,* d_T *and* d_s*, respectively, that are created by choosing a random* $\gamma \in \operatorname{End} E_0$ *of degree* $d_s^2 d_T$ *and decomposing it as in Figure 1. We assume that they are indistinguishable from random isogenies of their respective degrees and starting curves.*

Due to the fast mixing properties of isogeny graphs and since $d_s \approx p$ we can reach almost every j-invariant with an isogeny of degree d_s. Therefore, we should be able to find an $\psi \colon E_0 \to E_T$ of degree d_s for arbitrary $\varphi_T \circ \varphi_s \colon E_0 \to E_T$ of degree $d_s d_T$. This justifies Assumption 5.14 and allows us to prove the following theorem:

Theorem 5.15. *For a* $\frac{3}{2}\kappa$*-inverse resistant function* F *and under the Assumptions 5.5, 5.6, 5.8 and 5.14, SIGNITC is* (t_p, t_o, ε)*-CCA-secure (satisfies hiding) with security game from Definition 5.13 for* $t_p \ll 2^\kappa$ *polynomial in* κ*,* $t_o < t_{\text{fd}}$ *and* $\varepsilon = 2^{-\kappa}$.

Proof. For this proof "time" always means parallel time with $O(t_{\text{fd}})$ cores. Recall that $t_{\text{fd}} \in \text{poly}(\kappa)$ and especially $t_{\text{fd}} < 2^{\kappa/2}$. The precomputation phase can only provide a negligible advantage for an adversary $\mathcal{A}$. The computation of $\texttt{Com}(\mathbf{crs}, m)$ includes choosing random $\gamma \in \operatorname{End} E_0$ of degree $d_s^2 d_T$ and (implicitly) decomposing it into isogenies φ_s, φ_T and ψ of degrees d_s, d_T and d_s, respectively. Assumption 5.14 lets us treat them as random isogenies. Since there are $d_s \geq 2^{2\kappa}$ different isogenies $\varphi_s \colon E_0 \to E_s$ of degree d_s, it is infeasible to precompute (and store) a significant subset of all possibilities in time $t_p \ll 2^\kappa$ polynomial in κ.

In the online phase $\mathcal{A}$ sends two messages m_0, m_1 and receives the output (E_s, K_T, u_b) of $\texttt{Com}(\mathbf{crs}, m_b)$ for a uniform $b \in \{0, 1\}$. The adversary $\mathcal{A}$ knows

that $F(j_T)$ is equal to $F_0 = \ominus u_b \oplus m_0$ or $F_1 = \ominus u_b \oplus m_1$. Since F is a $\frac{3}{2}\kappa$-inverse resistant function, there are at least $2^{3\kappa/2}$ j-invariants j such that $F(j) = F_i$ for each $i \in \{0, 1\}$ and none of them is more likely than the other. To verify one of them, $\mathcal{A}$ would have to compute $E_s/\langle K_T \rangle$, but this is equivalent to computing $\texttt{FDecom}(\mathbf{crs}, (E_s, K_T, u_b))$. Under Assumptions 5.5, 5.6 and 5.8 we get that it can not be done in time t_o less than t_{fd} by Corollary 5.10.

We can try to find a neighbor j of j_T in the 2-isogeny graph. Then for the 3 neighbors j_1, j_2, j_3 of j we check if $F(j_k)$ matches F_0 or F_1. If we have only one match, then this gives $F(j_T)$. If we have more matches, then we have to try again with a different j. The problem with this approach is that computing j and its 3 neighbors is slower than computing $E_s/\langle K_T \rangle$. As discussed above, we assume that this can not be done in time t_o less than t_{fd}.

Another approach is to find E and $K \in E$ such that $E/\langle K \rangle$ is a neighbor of E_T in the 2-isogeny graph. If we query the oracle on $(E, K, 0)$ instead of computing $E/\langle K \rangle$, then we only get $F(j) = 0 \oplus F(j)$ instead of $j = j(E/\langle K \rangle)$. Since F is an $\frac{3}{2}\kappa$-inverse resistant function, there are at least $2^{3\kappa/2}$ indistinguishable candidates for each j. In this case the best approach is to find 3 pairs (E_k, K_k) such that $j_k = j(E_k/\langle K_k \rangle)$ are the 3 neighbors of j_T in the 2-isogeny graph. Then we query the oracle on all of them to get $F(j_k)$ for $1 \leq k \leq 3$. Now we have to match them with the neighbors of the (at least) $2^{3\kappa/2}$ candidates for j_T from each F_0 and F_1. To have confidence in a candidate, we have to compare its neighbors to several or all $F(j_k)$. If $F_0 \neq F_1$, then their preimages are disjoint and there are at least $2 \cdot 2^{3\kappa/2}$ candidates for j_T. Since $t_o < t_{\text{fd}} < 2^{\kappa/2}$ and $t_{\text{fd}} \in \text{poly}(\kappa)$ the total number of operations that can be done in parallel time t_o with $O(t_{\text{fd}})$ cores is $2^{o(\kappa)} < 2^{\kappa/2}$. Therefore, the probability to find j_T is less than $2^{\kappa/2} \cdot 2^{-3\kappa/2} = 2^{-\kappa}$. Applying Grover's algorithm [25] only helps with matching, but the more than $2^{3\kappa/2}$ candidates for j_T first have to be computed classically.

When we replace E_s and K_T by a curve E' and point K' such that $E'/\langle K' \rangle$ is unrelated to $E_s/\langle K_T \rangle$, the query $\texttt{FDecom}(\mathbf{crs}, (E', K', u_b))$ gives completely unrelated results. Guessing j_T correctly has a chance of roughly $12/p \approx 2^{-2\kappa}$ and is therefore negligible. In conclusion, in the online phase the advantage over guessing is less than $2^{-\kappa}$ under Assumptions 5.5, 5.6 and 5.8. □

The proof for binding works with the original Definition 2.6 and security game from Definition 2.5. With our protocol we even achieve perfect binding.

Theorem 5.16. *SIGNITC is $(\infty, 0)$-BND-CCA-secure (satisfies binding) with security game from Definition 2.5.*

Proof. If the commitment $\mathbf{C}$ is accepted by `ComVrfy`, then it contains an elliptic curve E_s, a point K_T on E_s and an element u of an additive group M. `DecVrfy` computes an ideal I_{sT} that corresponds to

$$\varphi_T \circ \varphi_s \colon E_0 \to E_s \to E_T \cong E_s/\langle K_T \rangle,$$

as well as an equivalent ideal I_ψ and an isogeny $\psi \colon E_0 \to \widetilde{E}_T$ corresponding to I_ψ. Since I_{sT} and I_ψ are equivalent their right orders are isomorphic and the codomains of the corresponding isogenies have to agree up to Galois conjugacy. F is oblivious to that and therefore we have $F(j_T) = F(\widetilde{j}_T)$ where j_T and $\widetilde{j}_T$ are the j-invariants of E_T and $\widetilde{E}_T$, respectively. Hence, `DecVrfy` verifies

$u \oplus F(\widetilde{j}_T) = u \oplus F(j_T) = m$. Thus, acceptance of both $(\mathbf{crs}, \mathbf{C}, m, \pi_{\text{dec}})$ and $(\mathbf{crs}, \mathbf{C}, m', \pi'_{\text{dec}})$ by `DecVrfy` implies $m \ominus F(j_T) = u = m' \ominus F(j_T)$ and hence $m = m'$. Similarly, if `DecVrfy` accepts $(\mathbf{crs}, \mathbf{C}, m, \pi_{\text{dec}})$ then $u = m \ominus F(j_T)$. `FDecom` computes $F(j_T)$ from E_s and K_T and thus outputs the correct message $m = u \oplus F(j_T)$. □

5.3 Practicality

We show that `Com`, `ComVrfy` and `DecVrfy` can be computed efficiently and that we achieve a perfectly practical NITC scheme. We chose $p \geq 2^{2\kappa} - 1$, $d_s \geq 2^{2\kappa}$, $d_s \approx p$ and $t_{\text{fd}} < 2^{\kappa/2}$ to get κ bits of classical and $\kappa/2$ bits of quantum security. In this subsection "efficiently" means an expected running time of at most $\text{poly}(\log p)$ operations for probabilistic algorithms.

Lemma 5.17. *The commitment* `Com` *takes time* $t_{\text{com}} \in \text{poly}(\log p, \log d_T)$.

Proof. The algorithm `FullRepresentInteger` and the element β can be computed in time $\text{poly}(\log p)$. Finding the ideals I_s, I'_T, I_ψ and their corresponding kernel generators $K_s, K'_T, K_\psi \in E_0$ can be done in time $\text{poly}(\log d_s)$, $\text{poly}(\log d_T)$ and $\text{poly}(\log d_s)$, respectively. By Lemma 5.4 we can find $E_s \cong E_0/\langle K_s \rangle$, $K_T = \varphi_s(K'_T)$ and $\widetilde{E}_T \cong E_0/\langle K_\psi \rangle$ via Vélu's formulae in time $O((\log d_s)^2)$ since d_s is smooth. Finally, we have to compute $\widetilde{j}_T = j(\widetilde{E}_T)$ and $u = m \ominus F(\widetilde{j}_T)$. Since we chose F and the group operation in M to be efficiently computable and $d_s \approx p$, we get that the algorithm takes time $t_{\text{com}} \in \text{poly}(\log p, \log d_T)$. □

Lemma 5.18. *The maximal number of operations* t_{cv} *for algorithm* `ComVrfy` *is in* $O(\log p, \log d_T)$.

Proof. The algorithm has to complete four tasks. First it has to check if E_s is an elliptic curve. To do that, it suffices to check that the discriminant is non-zero. For curves in short Weierstraß form $E\colon y^2 = x^3 + Ax + B$ this is just $4A^3 \neq -27B^2$. To check if K_T is a point on E_s it can simply compute if K_T satisfies the curve equation. Verifying $\operatorname{ord} K_T = d_T$ can be done by a few scalar multiplications in time $O(\log d_T)$. Finally, membership testing for $u \in M$ is efficient by definition of M. This would give us $\text{poly}(\log p)$, but for $M = \mathbb{Z}/N\mathbb{Z}$ this means checking if u is an integer (and if $0 \leq u < N$). For $N < p$ this only takes time $O(\log p)$. So all this can be done in $O(\log p, \log d_T)$ operations. □

Lemma 5.19. *The decommitment verification algorithm* `DecVrfy` *takes time* $t_{\text{dv}} \in \text{poly}(\log p, \log d_T)$.

Proof. The decommitment verification has basically the same steps as the commitment. There are only two differences: Firstly, it gets γ, β from π_{dec} instead of computing them. Secondly, it has to compare the E_s and K_T it computes to the ones in the commitment and m to the decommitment. Since these differences are computationally insignificant we get that the algorithm also takes time $t_{\text{dv}} \in \text{poly}(\log p, \log d_T)$. □

Note that the running times of `Com`, `ComVrfy` and `DecVrfy` only depend logarithmically on d_T, whereas `FDecom` takes parallel time at least $t_{\text{fd}} = d_T^{0.284}$. So we can choose d_T such that $t_{\text{com}}, t_{\text{cv}}, t_{\text{dv}} \ll t_{\text{fd}}$.

Theorem 5.20. *SIGNITC is perfectly practical under Assumptions 5.5, 5.6 and 5.8.*

6 Example Parameters

As mentioned in Section 4.3 we have wide variety of primes to choose from. On our GitHub[8] we list all parameter sets we used. We tested five different primes with suitable factors d_T of $p^2 - 1$. In order of their size they are: the Mersenne prime $p_{M_{127}} = 2^{127} - 1$, a prime p_{2D} from [30], a prime p_{1973} from the specifications of SQIsign [13], a prime p_{6983} from the SQISign paper [19] and another Mersenne prime $p_{M_{521}} = 2^{521} - 1$. In Table 1 we present the running times of the different algorithms of SIGNITC on a single Intel Core i5-6500 @3.20 GHz with 8 GB memory and 6 MiB cache. Computing isogenies of larger prime degree requires more memory. Therefore, `FDecom` was not computed for larger d_T. In the corresponding column we wrote "? unit" to represent the expected delay, e.g. "? days" for "several days".

prime	κ	$\log_2 d_T$	`Com`	`ComVrfy`	`DecVrfy`	`FDecom`
$p_{M_{127}}$	127	36.2	1.14 s	109 ms	786 ms	19.8 min
p_{2D}	250	45.2	2.74 s	251 ms	2.06 s	? days
p_{1973}	252	28.0	2.72 s	158 ms	2.32 s	1.23 min
p_{6983}	255	38.9	2.81 s	161 ms	2.33 s	? hours
p_{6983}	255	44.6	2.61 s	160 ms	2.32 s	? days
$p_{M_{521}}$	521	25.0	10.9 s	1.75 s	9.24 s	42.0 s
$p_{M_{521}}$	521	28.2	9.97 s	1.75 s	8.09 s	2.03 min
$p_{M_{521}}$	521	31.2	10.3 s	1.76 s	9.15 s	6.98 min
$p_{M_{521}}$	521	35.9	10.6 s	1.78 s	9.02 s	46.1 min
$p_{M_{521}}$	521	46.6	10.9 s	1.76 s	9.07 s	? months
$p_{M_{521}}$	521	47.0	9.58 s	1.81 s	9.03 s	? months

Table 1: Sequential running times of SIGNITC for different parameter sets. A question mark denotes expected time.

We can see that `Com`, `ComVrfy` and `DecVrfy` are dominated by κ or rather $\log p \approx \log d_s$ and do not change significantly for different d_T. `FDecom` on the other side depends heavily on d_T. If the result γ of `FullRepresentInteger` happens to be in $\mathbb{Z}[i, j] \subset \mathcal{O}_0$, the algorithms `Com` and `ComVrfy` are significantly faster. This shows that SIGNITC can be implemented with realistic parameters. It also shows that post-quantum isogeny-based delay is possible.

Conclusion

We showed that SIGNITC is a perfectly practical NITC that satisfies hiding and perfect binding. It is the first NITC without repeated squaring or black box algorithms and it needs no trusted setup. It has an implementation in SageMath working with realistic parameters. Since it uses only isogeny-based cryptography, it is presumably quantum secure. This could also be an interesting starting point for isogeny-based delay in other settings.

Finally, we list some open topics for further research. The most obvious ones are to test larger delays and to implement optimizations. These include using

[8] GITHUB

a more efficient programming language than SageMath, adding parallelization and removing redundancy due to the use of generic algorithms.

Another open question is how this can be implemented efficiently using delay isogenies of smooth degree. This requires dealing with extension fields. A recent paper [10] introduces new algorithms that can potentially improve computations involving higher extension degrees. We only looked at the number of operations regardless of the field extensions. Since working in higher extension fields can significantly slow down the computations, it might be beneficial to represent our isogenies other than with one point in a (possibly large) extension field in this case.

Acknowledgments

The author would like to thank Antonio Sanso for pointing out the concept of NITC schemes, Valerie Gilchrist, Lorenz Panny and anonymous reviewers for their comments on previous versions of this article, Max Duparc for interesting discussions on pushforwards and higher dimensional isogenies, and Jens Zumbrägel for his support during the work on this article.

References

[1] Knud Ahrens and Jens Zumbrägel. DEFEND: Towards verifiable delay functions from endomorphism rings. Cryptology ePrint Archive, Paper 2023/1537, 2023. URL `https://eprint.iacr.org/2023/1537`.

[2] Miguel Ambrona, Marc Beunardeau, and Raphaël R. Toledo. Timed commitments revisited. Cryptology ePrint Archive, Paper 2023/977, 2023. URL `https://eprint.iacr.org/2023/977`.

[3] Andrea Basso, Giulio Codogni, Deirdre Connolly, Luca De Feo, Tako Boris Fouotsa, Guido Maria Lido, Travis Morrison, Lorenz Panny, Sikhar Patranabis, and Benjamin Wesolowski. Supersingular curves you can trust. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023*, pages 405–437, Cham, 2023. Springer Nature Switzerland.

[4] Andrea Basso, Pierrick Dartois, Luca De Feo, Antonin Leroux, Luciano Maino, Giacomo Pope, Damien Robert, and Benjamin Wesolowski. SQIsign2D–West. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 339–370, Singapore, 2025. Springer Nature Singapore. ISBN 978-981-96-0891-1. doi: 10.1007/978-981-96-0891-1_11.

[5] Daniel J. Bernstein, Luca De Feo, Antonin Leroux, and Benjamin Smith. Faster computation of isogenies of large prime degree. In Steven D. Galbraith, editor, *Proceedings of the Fourteenth Algorithmic Number Theory Symposium*, pages 39–55, Berkeley, 2020. Mathematical Sciences Publishers. doi: 10.2140/obs.2020.4.39.

[6] Alex Biryukov, Ben Fisch, Gottfried Herold, Dmitry Khovratovich, Gaëtan Leurent, María Naya-Plasencia, and Benjamin Wesolowski. Cryptanalysis of algebraic verifiable delay functions. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024*, pages 457–490, Cham, 2024. Springer Nature Switzerland. ISBN 978-3-031-68382-4. doi: 10.1007/978-3-031-68382-4_14.

[7] Dan Boneh and Moni Naor. Timed commitments. In Mihir Bellare, editor, *Advances in Cryptology – CRYPTO 2000*, pages 236–254, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg. doi: 10.1007/3-540-44598-6_15.

[8] Dan Boneh, Joseph Bonneau, Benedikt Bünz, and Ben Fisch. Verifiable delay functions. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018*, pages 757–788, Cham, 2018. Springer International Publishing. doi: 10.1007/978-3-319-96884-1_25.

[9] Jeffrey Burdges and Luca De Feo. Delay encryption. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021*, pages 302–326, Cham, 2021. Springer International Publishing. ISBN 978-3-030-77870-5. doi: 10.1007/978-3-030-77870-5_11.

[10] Shiping Cai, Mingjie Chen, and Christophe Petit. Faster algorithms for isogeny computations over extensions of finite fields. In Andrzej Dąbrowski, Josef Pieprzyk, and Jacek Pomykała, editors, *Number-Theoretic Methods in Cryptology*, pages 136–155, Cham, 2025. Springer Nature Switzerland. doi: 10.1007/978-3-031-82380-0_4.

[11] Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. CSIDH: An efficient post-quantum commutative group action. In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018*, pages 395–427, Cham, 2018. Springer International Publishing. doi: 10.1007/978-3-030-03332-3_15.

[12] Jorge Chavez-Saab, Francisco Rodríguez-Henríquez, and Mehdi Tibouchi. Verifiable isogeny walks: Towards an isogeny-based postquantum VDF. In Riham AlTawy and Andreas Hülsing, editors, *Selected Areas in Cryptography*, pages 441–460, Cham, 2022. Springer International Publishing. doi: 10.1007/978-3-030-99277-4_21.

[13] Jorge Chavez-Saab, Maria Corte-Real Santos, Luca De Feo, Jonathan Komada Eriksen, Basil Hess, David Kohel, Antonin Leroux, Patrick Longa, Michael Meyer, Lorenz Panny, Sikhar Patranabis, Christophe Petit, Francisco Rodríguez Henríquez, Sina Schaeffler, and Benjamin Wesolowski. SQIsign algorithm specifications and supporting documentation. Project Homepage, 2023. URL `https://sqisign.org/spec/sqisign-20230601.pdf`.

[14] Peter Chvojka and Tibor Jager. Simple, fast, efficient, and tightly-secure non-malleable non-interactive timed commitments. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *Public-Key Cryptography – PKC 2023*, pages 500–529, Cham, 2023. Springer Nature Switzerland. doi: 10.1007/978-3-031-31368-4_18.

[15] Jorge Chávez-Saab, Odalis Ortega, and Amalia Pizarro-Madariaga. On the parallelization of square-root vélu's formulas. *Mathematical and Computational Applications*, 29(1), 2024. doi: 10.3390/mca29010014. URL `https://www.mdpi.com/2297-8747/29/1/14`.

[16] Pierrick Dartois, Antonin Leroux, Damien Robert, and Benjamin Wesolowski. SQIsignHD: New dimensions in cryptography. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024*, pages 3–32, Cham, 2024. Springer Nature Switzerland. ISBN 978-3-031-58716-0. doi: 10.1007/978-3-031-58716-0_1.

[17] Luca De Feo. Mathematics of isogeny based cryptography. Preprint, 2017. URL `https://arxiv.org/abs/1711.04062`.

[18] Luca De Feo, Simon Masson, Christophe Petit, and Antonio Sanso. Verifiable delay functions from supersingular isogenies and pairings. In Steven D. Galbraith and Shiho Moriai, editors, *Advances in Cryptology – ASIACRYPT 2019*, pages 248–277, Cham, 2019. Springer International Publishing. doi: 10.1007/978-3-030-34578-5_10.

[19] Luca De Feo, David Kohel, Antonin Leroux, Christophe Petit, and Benjamin Wesolowski. SQISign: Compact post-quantum signatures from quaternions and isogenies. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020*, pages 64–93, Cham, 2020. Springer International Publishing. doi: 10.1007/978-3-030-64837-4_3.

[20] Luca De Feo, Antonin Leroux, Patrick Longa, and Benjamin Wesolowski. New algorithms for the deuring correspondence. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023*, pages 659–690, Cham, 2023. Springer Nature Switzerland. doi: 10.1007/978-3-031-30589-4_23.

[21] Thomas Decru, Luciano Maino, and Antonio Sanso. Towards a quantum-resistant weak verifiable delay function. In Abdelrahaman Aly and Mehdi Tibouchi, editors, *Progress in Cryptology – LATINCRYPT 2023*, pages 149–168, Cham, 2023. Springer Nature Switzerland. doi: 10.1007/978-3-031-44469-2_8.

[22] Max Deuring. Die Typen der Multiplikatorenringe elliptischer Funktionenkörper. *Abh. Math. Sem. Hansischen Univ.*, 14:197–272, 1941. doi: 10.1007/BF02940746.

[23] Kirsten Eisenträger, Sean Hallgren, Kristin Lauter, Travis Morrison, and Christophe Petit. Supersingular isogeny graphs and endomorphism rings: Reductions and solutions. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018*, pages 329–368, Cham, 2018. Springer International Publishing. doi: 10.1007/978-3-319-78372-7_11.

[24] Jonathan Komada Eriksen, Lorenz Panny, Jana Sotáková, and Mattia Veroni. Deuring for the people: Supersingular elliptic curves with prescribed endomorphism ring in general characteristic. Cryptology ePrint Archive, Paper 2023/106, 2023. URL `https://eprint.iacr.org/2023/106`.

[25] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, STOC '96, pages 212–219, New York, NY, USA, 1996. Association for Computing Machinery. doi: 10.1145/237814.237866.

[26] David Jao, Reza Azarderakhsh, Matthew Campagna, Craig Costello, Luca De Feo, Basil Hess, Amir Jalali, Brian Koziel, Brian LaMacchia, Patrick Longa, Michael Naehrig, Geovandro Pereira, Joost Renes, Vladimir Soukharev, and David Urbanik. Supersingular isogeny key encapsulation. Project Homepage, 2020. URL `https://sike.org/files/SIDH-spec.pdf`.

[27] Jonathan Katz, Julian Loss, and Jiayu Xu. On the security of time-lock puzzles and timed commitments. In Rafael Pass and Krzysztof Pietrzak, editors, *Theory of Cryptography*, pages 390–413, Cham, 2020. Springer International Publishing. doi: 10.1007/978-3-030-64381-2_14.

[28] David Kohel, Kristin Lauter, Christophe Petit, and Jean-Pierre Tignol. On the quaternion ℓ-isogeny path problem. *LMS J. Comput. Math.*, 17:418–432, 2014. doi: 10.1112/S1461157014000151.

[29] Antonin Leroux. Verifiable random function from the deuring correspondence and higher dimensional isogenies. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, pages 167–194, Cham, 2025. Springer Nature Switzerland. doi: 10.1007/978-3-031-91098-2_7.

[30] Hiroshi Onuki and Kohei Nakagawa. Ideal-to-isogeny algorithm using 2-dimensional isogenies and its application to sqisign. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 243–271, Singapore, 2025. Springer Nature Singapore. ISBN 978-981-96-0891-1. doi: 10.1007/978-981-96-0891-1_8.

[31] Christophe Petit and Spike Smith. An improvement to the quaternion analogue of the ℓ-isogeny path problem. MathCrypt 2018, 2018. URL `https://crypto.iacr.org/2018/affevents/mathcrypt/medias/08-50_3.pdf`.

[32] Ronald L. Rivest, Adi Shamir, and David A. Wagner. Time-lock puzzles and timed-release crypto. Technical report, USA, 1996.

[33] Joseph H. Silverman. *The Arithmetic of Elliptic Curves*, volume 106 of *Graduate Texts in Mathematics*. Springer New York, 1986. doi: 10.1007/978-1-4757-1920-8.

[34] Bruno Sterner. Commitment schemes from supersingular elliptic curve isogeny graphs. *Transactions on Mathematical Cryptology*, 1(2):40–51, Mar. 2022. URL `https://journals.flvc.org/mathcryptology/article/view/130656`.

[35] Sri Aravinda Krishnan Thyagarajan, Guilhem Castagnos, Fabian Laguillaumie, and Giulio Malavolta. Efficient CCA timed commitments in class groups. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, CCS '21, page 2663–2684, New York, NY, USA, 2021. Association for Computing Machinery. doi: 10.1145/3460120.3484773.

[36] John Voight. *Quaternion algebras*, volume 288 of *Graduate Texts in Mathematics*. Springer Cham, 2021. doi: 10.1007/978-3-030-56694-4.

[37] Jacques Vélu. Isogénies entre courbes elliptiques. *Comptes Rendus Hebdomadaires des Séances de l'Académie des Sciences, Série A*, 273, N°4:238–241, 1971.

[38] Benjamin Wesolowski. The supersingular isogeny path and endomorphism ring problems are equivalent. In *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1100–1111, 2022. doi: 10.1109/FOCS52979.2021.00109.